

# Ejercicio Demostrativo

---

Sergio Sebastian Sandoval Ruiz

**Tiempo estimado: 25 min**

Universidad San Carlos de Guatemala  
Facultad de Ingeniería  
Escuela de Ingeniería en Ciencias y Sistemas.  
Segundo Semestre 2026  
[Laboratorio - Estructura de Datos - C]

# 1. Marco Formativo

## 1.1 Valores

|                      |                                                                                                                                                                                                                                                                                                                                |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Perseverancia</b> | Los ejercicios de eliminación en AVL son el punto más difícil del tema: requieren eliminar como en BST, luego recalculan todos los factores de equilibrio hacia la raíz y aplicar rotaciones en cascada si es necesario. El estudiante que persiste y no abandona el trazado manual logra dominar la estructura completamente. |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 1.2 Competencias

|                               |                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tipo de Competencia           | Redacción                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Competencia General</b>    | Maneja grandes volúmenes de datos aplicando árboles para optimizar búsquedas.                                                                                                                                                                                                                                                                                                                       |
| <b>Competencia Específica</b> | Aplica la operación de eliminación en un BST manejando correctamente los tres casos (nodo hoja, un hijo, dos hijos con sucesor inorden); ejecuta eliminación en un AVL recalculando el factor de equilibrio en todos los ancestros afectados y aplicando las rotaciones necesarias en cascada para restaurar el balanceo; y traza los tres recorridos sobre árboles más complejos de 8 o más nodos. |

## 2. Palabras Clave

**Eliminación en BST — Caso 1 (hoja):** El nodo a eliminar no tiene hijos. Se elimina directamente desconectando el puntero del padre que apuntaba a él.

**Eliminación en BST — Caso 2 (un hijo):** El nodo a eliminar tiene exactamente un hijo. El padre del nodo eliminado adopta al único hijo como nuevo hijo directo.

**Eliminación en BST — Caso 3 (dos hijos):** El nodo a eliminar tiene dos hijos. Se reemplaza su valor por el del sucesor in-orden (mínimo del subárbol derecho) o el predecesor in-orden (máximo del subárbol izquierdo), y luego se elimina ese sucesor/predecesor que tendrá como máximo un hijo.

**Sucesor in-orden:** El nodo con el valor inmediatamente mayor al nodo actual dentro del BST. Se encuentra bajando una vez a la derecha y luego todo lo posible a la izquierda.

**Predecesor in-orden:** El nodo con el valor inmediatamente menor al nodo actual. Se encuentra bajando una vez a la izquierda y luego todo lo posible a la derecha.

**Rotación en cascada:** Situación en un AVL donde una sola eliminación provoca desbalanceo en más de un nodo ancestro, requiriendo múltiples rotaciones desde el nodo eliminado hasta la raíz.

**Recálculo de factores tras eliminación:** Después de eliminar en AVL, se debe recalcular el factor de equilibrio de cada ancestro del nodo eliminado, de abajo hacia arriba, aplicando rotación cuando el factor excede  $\pm 1$ .

**Árbol degenerado:** BST que se comporta como lista enlazada porque todas las inserciones se realizaron en orden estrictamente creciente o decreciente. Altura =  $n$ , búsqueda  $O(n)$ .

### 3. Resumen

Esta sesión de refuerzo profundiza en el tema de árboles BST y AVL iniciado en la Semana 7, con énfasis en la operación de eliminación — la más compleja de ambas estructuras. El estudiante ya conoce la teoría; ahora debe aplicarla en ejercicios con árboles de mayor tamaño, múltiples rotaciones consecutivas y trazados de recorridos sobre estructuras que no están preconstruidas en la diapositiva.

La clase se orienta a que el estudiante domine los tres casos de eliminación en BST, comprenda cómo la eliminación en AVL puede desencadenar rotaciones en múltiples niveles del árbol, y consolide los recorridos Pre-Orden, In-Orden y Post-Orden sobre árboles contruidos en vivo durante la sesión.

## 4.Contenido

### 4.1 DESCRIPCIÓN DEL EJERCICIO

El ejercicio consiste en construir un BST con la secuencia [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45] y demostrar la eliminación de tres nodos: uno hoja (10), uno con un hijo (20), y uno con dos hijos (30). Luego se construye un AVL con la secuencia [15, 10, 20, 5, 12, 17, 25, 3, 8] y se demuestra la eliminación del nodo 20, observando cómo el rebalanceo puede propagarse hacia la raíz.

### 4.2 OBJETIVO DE LA DEMOSTRACIÓN

Demostrar el procedimiento completo de eliminación en BST y en AVL, manejando los tres casos posibles y el rebalanceo en cascada, con el fin de que el estudiante comprenda que la eliminación en AVL es idéntica a la de BST en su primera fase pero requiere una segunda fase de verificación y corrección del factor de equilibrio en todos los ancestros afectados, de abajo hacia arriba.

### 4.3 RECURSOS NECESARIOS

- **Software:** Java JDK 11+, IDE (IntelliJ IDEA / Eclipse / VS Code)
- **Herramientas técnicas:** Compilador javac, terminal/consola
- **Materiales de apoyo:** Diapositivas Semana 7/8, pizarrón, visualizador online <https://visualgo.net/en/bst>
- **Archivos:** Archivo base AVL.java con el código de la Semana 7 ya implementado (inserciones y rotaciones)
- **Configuración mínima:** Java 11+, 4 GB RAM, proyector

### 4.4 DESARROLLO PASO A PASO

#### Paso 1 — Repaso rápido de recorridos sobre árbol grande (10 min)

Construir en pizarrón el BST con [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45]. Pedir a estudiantes que den el resultado de los tres recorridos sin calcularlo el auxiliar:

- **Pre-Orden esperado:** 50 30 20 10 25 40 35 45 70 60 80
- **In-Orden esperado:** 10 20 25 30 35 40 45 50 60 70 80 ← siempre ordenado

- **Post-Orden esperado:** 10 25 20 35 45 40 30 60 80 70 50

**Qué observar:** In-Orden da el arreglo [10, 20, 25, 30, 35, 40, 45, 50, 60, 70, 80] — ordenado ascendentemente, confirmando que el BST está correcto.

---

## Paso 2 — Eliminación BST Caso 1: nodo hoja (10 min)

Eliminar el nodo **10** (hoja, sin hijos):

java

Java

```
NodoBST eliminar(NodoBST nodo, int valor) {
    if (nodo == null) return null;
    if (valor < nodo.dato)
        nodo.izquierdo = eliminar(nodo.izquierdo, valor);
    else if (valor > nodo.dato)
        nodo.derecho = eliminar(nodo.derecho, valor);
    else {
        // CASO 1: sin hijos
        if (nodo.izquierdo == null && nodo.derecho == null)
            return null;
        // CASO 2: un hijo
        if (nodo.izquierdo == null) return nodo.derecho;
        if (nodo.derecho == null) return nodo.izquierdo;
        // CASO 3: dos hijos - reemplazar con sucesor in-orden
        NodoBST sucesor = minimoNodo(nodo.derecho);
        nodo.dato = sucesor.dato;
        nodo.derecho = eliminar(nodo.derecho, sucesor.dato);
    }
    return nodo;
}

NodoBST minimoNodo(NodoBST nodo) {
```

```
while (nodo.izquierdo != null) nodo = nodo.izquierdo;
return nodo;
}
```

**Qué observar:** Al eliminar el 10 (hoja), el nodo 20 queda con solo el hijo derecho (25). El árbol sigue siendo un BST válido.

---

### Paso 3 — Eliminación BST Caso 2: nodo con un hijo (10 min)

Eliminar el nodo **20** (ahora solo tiene hijo derecho 25 tras eliminar el 10):

**Qué observar:** El padre de 20 (que es 30) ahora apunta directamente a 25. Verificar en pizarrón que la propiedad BST se mantiene:  $25 < 30$ , está bien en el subárbol izquierdo de 30.

---

### Paso 4 — Eliminación BST Caso 3: nodo con dos hijos (15 min)

Eliminar el nodo **30** (tiene hijos 25 a la izquierda y 40 a la derecha):

#### Procedimiento:

1. El sucesor in-orden de 30 es el mínimo del subárbol derecho → bajar a 40, luego todo a la izquierda → llega a **35**.
2. Reemplazar el valor de 30 por 35.
3. Eliminar el nodo que tenía el valor 35 (que es hoja → Caso 1).

**Resultado:** La raíz del subárbol izquierdo de 50 ahora es 35, con hijos 25 (izq) y 40 (der, que a su vez tiene hijo derecho 45).

**Qué observar:** El valor 35 aparece ahora en la posición donde estaba 30. In-Orden ahora da **[25, 35, 40, 45, 50, 60, 70, 80]** — sigue ordenado.

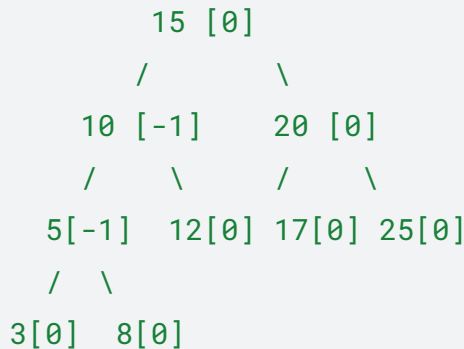
---

### Paso 5 — Eliminación en AVL con rebalanceo (25 min)

Construir el AVL con **[15, 10, 20, 5, 12, 17, 25, 3, 8]** y verificar que está balanceado. Luego eliminar el nodo **20**:

None

Estado antes de eliminar 20:

**Procedimiento:**

1. El nodo 20 tiene dos hijos → sucesor in-orden es **25** (mínimo de su subárbol derecho, que es hoja).
2. Reemplazar 20 por 25, eliminar el nodo que tenía 25 (hoja).
3. Recalcular factores de equilibrio hacia arriba desde la posición del nodo eliminado:
  - Nodo 25 (nuevo, reemplazó a 20): factor = altura(der) – altura(izq) = 0 – 1 = **-1** ✓
  - Nodo 15: factor = altura(der) – altura(izq) = 2 – 3 = **-1** ✓ (no requiere rotación)

**Resultado:** El árbol queda balanceado sin necesidad de rotación en este caso. Mostrar el árbol resultante y verificar los factores.

**Caso adicional para demostrar rotación en cascada (en pizarrón):** Partir del mismo AVL y eliminar el nodo **17** en su lugar. Tras la eliminación el nodo 20 (ahora solo con hijo 25) queda con factor -1. El nodo 15 queda con factor -2 → se requiere **RD** (o RDI según el factor del hijo). Demostrar la rotación.

**Observaciones Importantes**

- En el Caso 3 de eliminación, tanto el sucesor in-orden como el predecesor in-orden son válidos para reemplazar el nodo. La convención más común es usar el sucesor (mínimo del subárbol derecho) pero ambos producen un BST válido.
- En AVL, la eliminación puede provocar rotaciones en cascada: a diferencia de la inserción que requiere como máximo una rotación simple o doble, la eliminación puede requerir rotaciones en múltiples niveles hasta llegar a la raíz.
- Un error muy frecuente es olvidar recalcular el factor de equilibrio en **TODOS** los ancestros del nodo eliminado, no solo en el padre inmediato.
- La regla para identificar rotación tras eliminación es la misma que tras inserción: mirar el factor del nodo desbalanceado y el factor de su hijo más alto.
-

#### 4.5 Tabla Comparativa — Pila vs Cola

##### TABLA COMPARATIVA

| Operación                      | BST<br>(promedio) | BST (peor<br>caso) | AVL<br>(siempre) | Observación                     |
|--------------------------------|-------------------|--------------------|------------------|---------------------------------|
| Eliminar nodo<br>hoja          | $O(\log n)$       | $O(n)$             | $O(\log n)$      | Más simple, solo<br>desconectar |
| Eliminar nodo 1<br>hijo        | $O(\log n)$       | $O(n)$             | $O(\log n)$      | El padre adopta al<br>nieto     |
| Eliminar nodo 2<br>hijos       | $O(\log n)$       | $O(n)$             | $O(\log n)$      | Buscar sucesor +<br>reemplazar  |
| Rotaciones tras<br>eliminación | Ninguna           | Ninguna            | 0 a $O(\log n)$  | AVL puede rotar en<br>cascada   |
| Altura<br>garantizada          | No                | $O(n)$             | $O(\log n)$      | Ventaja clave del<br>AVL        |

#### 4.6 Resultado Esperado

| Operación                                                | Resultado                                                                                        |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Eliminar 10 (hoja) del BST                               | Nodo 20 queda solo con hijo derecho 25.<br>In-Orden:<br>[20, 25, 30, 35, 40, 45, 50, 60, 70, 80] |
| Eliminar 20 (un hijo) del BST                            | Nodo 30 adopta a 25 como hijo izquierdo.<br>In-Orden:<br>[25, 30, 35, 40, 45, 50, 60, 70, 80]    |
| Eliminar 30 (dos hijos) del BST                          | Nodo 30 reemplazado por sucesor 35.<br>In-Orden:<br>[25, 35, 40, 45, 50, 60, 70, 80]             |
| Eliminar 20 del AVL<br>[15, 10, 20, 5, 12, 17, 25, 3, 8] | Reemplazado por 25. Árbol queda balanceado sin rotación. Altura = 4                              |
| Eliminar 17 del AVL (ejercicio adicional)                | Nodo 15 queda con factor -2 → rotación requerida para rebalancear                                |
|                                                          |                                                                                                  |

#### 4.7 Aplicación Práctica

La eliminación eficiente en BST y AVL es crítica en sistemas de bases de datos: cada vez que se borra un registro indexado, el motor debe actualizar el árbol de índice manteniendo su propiedad de búsqueda. En MySQL por ejemplo, los índices B+ (extensión del BST) se rebalancean automáticamente al eliminar filas.

En el proyecto del curso (EDD MedTrack Fase 2), la operación de dar de baja un medicamento del inventario corresponde a una eliminación en el BST/AVL. Si el medicamento eliminado tiene tanto medicamentos con menor código como con mayor código registrados en el árbol, aplica el Caso 3 con sucesor in-orden.

## 5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- El Caso 3 de eliminación (dos hijos) es donde más se confunden los estudiantes, especialmente al encontrar el sucesor in-orden. Hay que insistir en que siempre se baja una vez a la derecha del nodo a eliminar y luego se va todo lo posible a la izquierda.
- En AVL, los estudiantes tienden a verificar solo el factor del padre inmediato del nodo eliminado y olvidan revisar los ancestros más arriba. Hay que enfatizar que la verificación sube hasta la raíz.
- Confunden la dirección de la rotación: al eliminar un nodo del subárbol izquierdo, el árbol queda desbalanceado hacia la derecha, lo que requiere RD (no RI). Siempre preguntar: ¿de qué lado quedó más pesado?

Recomendaciones:

- Dedicar los primeros 10 minutos a un repaso rápido de los recorridos con el árbol grande — esto sirve de calentamiento y verifica que los estudiantes recuerdan la Semana 7.
- Para la eliminación, usar dos colores en el pizarrón: uno para el nodo que se va a eliminar y otro para el sucesor que lo reemplaza. Ayuda a no perder de vista qué nodo se está moviendo.
- Mostrar visualmente en el visualizador online ([visualgo.net](http://visualgo.net)) la eliminación en AVL con cascada para que el estudiante vea las múltiples rotaciones animadas.
- Preparar 3 árboles preconstruidos (uno por cada caso de eliminación) en el pizarrón antes de que lleguen los estudiantes, para no perder tiempo dibujando durante la clase.

Puntos de mayor dificultad:

- Encontrar correctamente el sucesor in-orden en el Caso 3.
- Entender que al reemplazar el nodo con el sucesor, el sucesor in-orden siempre tiene como máximo un hijo (el hijo derecho), por lo que su eliminación nunca vuelve a ser Caso 3.

- El rebalanceo en cascada tras eliminación en AVL — el concepto de que la propagación sube hasta la raíz.

## 6. Referencias

1. GeeksforGeeks. (2023). Deletion in Binary Search Tree.  
<https://www.geeksforgeeks.org/deletion-in-binary-search-tree/>
2. GeeksforGeeks. (2023). AVL Tree Deletion.  
<https://www.geeksforgeeks.org/avl-tree-set-2-deletion/>
3. Visualizador interactivo BST y AVL: <https://visualgo.net/en/bst>
4. Cormen, T. H. et al. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Cap. 12.3: Eliminación en árboles binarios de búsqueda.
5. Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Pearson. Cap. 4.4: Eliminación en AVL.
6. Material de cátedra: Diapositivas Semana 8, EDD [B] — Árboles BST y AVL (Refuerzo). USAC, 2026.